

TEAM ATELIER CODING STANDARDS

Gavin Burgio, Bradley Duran, Caleb John, Kamare Sutherland

1. Purpose and Scope

- This document sets coding standards and best practices for JavaScript, CSS and HTML.
- Following them keeps code:
 - **High quality:** From start to finish, code should remain easy to understand, debug, and test.
 - **Maintainable:** Keep functions short and classes and modules small. Avoid adding complex designs.
 - **Consistent:** Every member should apply these coding conventions to maintain consistency across the project.

2. Naming Conventions

Key:

- camelCase = The first word starts with a small letter, and every next word starts with a capital letter
- PascalCase = The first word and every next word both start with a capital letter
- UPPER_SNAKE_CASE = All letters are uppercase and words are separated by underscores
- kebab-case = All spaces are replaced with hyphens and all letters are lowercase

Entity	Convention	Example
Variables, parameters, properties	camelCase, noun form	licensePlate, selectedUser
Functions and methods	camelCase, begins with a verb	addVehicle(), fetchUserList()
Classes, constructors, components	PascalCase, noun	CustomerService, UserCard
Constants	UPPER_SNAKE_CASE	MAX_MILEAGE, API_BASE_URL
Booleans	Prefix is, has, can, should	isVisible, hasError
Module / file names	kebab-case and ends in .js, .css, or .html	account-service.js
CSS classes	kebab-case, BEM: block__element--modifier	.card__title--large
CSS custom properties	-- kebab-case, grouped by role (e.g color, spacing)	--color-primary, --space-md
HTML ids, data attributes	kebab-case	id="main-nav", data-user-id

Note: All function and program variable names shouldn't exceed a character length of 10-15

3. JavaScript Coding Format and Practice

3.1 Layout

- **Indentation:** 2 spaces per nesting level; tabs shall not be used (tabs display at different widths in different editors). Configure the editor to insert 2 spaces when Tab is pressed. Files are UTF-8 with LF line endings. Lines should not exceed **100 characters**.
- **Quotes and semicolons:**
 - Every statement ends with a semicolon
 - JavaScript strings use **single quotes** (double quotes only when the text contains an apostrophe, e.g. "Don't save")

- HTML attributes use **double quotes**.
- Multi-line literals and argument lists end with a trailing comma.
- **Braces:** the opening brace is on the line of the statement; the closing brace is on its own line, aligned with the statement. Braces are required for every if, else, for and while, even for a single statement.
- One statement and one declaration per line. A blank line separates logical blocks; never two in a row.

3.2 Language Use

- **Variables:** const by default (unless you know you will reassign later, then use let)
- **Comparison:** Use === and !==
- **Exports:** Prefer named exports (export function x, import { x }) so every file uses the same name and renames are safe.
- **Imports:** Order of imports, first platform/built-ins, second Third-party packages and then project modules. Avoid circular imports.
- **Constants:** Replace magic numbers and repeated strings with named constants (SERVICE_INTERVAL_MILES, STATUS_IN_PROGRESS); 0, 1 and -1 with obvious meaning are fine.
- **Nesting:** Shall not exceed 3 levels, use early returns (return, or continue in loops) to handle the exit cases first.

3.3 Comments and Documentation

- Comments explain the function so that someone else can understand it's purpose without reading its body.
- Update comments when code changes and delete commented-out code.

Where	Syntax
JavaScript, short (outside JSX)	// comment, on the line above the code it describes
JavaScript, long / multi-line	/* comment */
Functions, classes, modules	/** JSDoc */
JSX markup (inside return), any length	{/* comment */}
CSS, any length	/* comment */
HTML, any length	<!-- comment -->

4. CSS Coding Format and Practice

- One selector per line and one declaration per line
- Indented 2 spaces
- Space after each colon
- Every declaration, including the last, ends with a semicolon
- Opening brace on the selector line, closing brace on its own line
- Lower case hex colors (e.g #ffffff)
- Style with classes (e.g .heading → <div class="heading">)

Use	Unit	Example	
Font sizes, padding, margins, gaps, component sizes	rem	font-size: 1rem; (scales with the user's browser text setting)	
Borders, outlines, hairlines	px	border: 1px solid ...;	
Line height	none	line-height: 1.5; (zero values also take no unit: margin: 0;)	

Use	Unit	Example	
Fluid layout sizes	%, vw, vh	width: 100%; min-height: 100vh;	

5. HTML Coding Format and Practice

- Begin with `<!doctype html>`, `<html lang="...">`, `<meta charset="utf-8">` and a viewport tag.
- Use **semantic elements** (header, nav, main, section, button) instead of generic div/span.
- One h1 per page and do not skip heading levels.
- `<button>` for actions
- `<a href>` for navigation.
- Every `<img/>` has alt; every form control has a `<label>`

6. Version Control, Reviews, Testing and Security

- **Version control:** regular commits to the team GitHub with related changes including a meaningful message and using branches for features, fixes and releases. Always pull from GitHub prior to working to have latest changes.
- **Code reviews:** at least one other team member reviews all code before merging into main, checking standards, logic errors and simplifications.
- **Testing:** unit tests for new modules were practical, in `*.test.js` beside the code; one suite per module. Prioritize meaningful tests over coverage numbers.
- **Security:** never insert untrusted data into HTML, URLs or queries; use parameterized queries; keep secrets out of source control; handle personal data per applicable regulations.

7. Conclusion

- All members of the team are expected to follow these coding standards. By doing so, the quality of the code space along with readability and collaboration will improve. In an event where a member wants to add something to the standards or discuss changing the standards, a meeting will be held to do so.